



LLM01:2025 Prompt Injection

A Prompt Injection Vulnerability occurs when user prompts alter the LLM's behavior or output in unintended ways. These inputs can affect the model even if they are imperceptible to humans, therefore prompt injections do not need to be human-visible/readable, as long as the content is parsed by the model.

Prompt Injection vulnerabilities exist in how models process prompts, and how input may force the model to incorrectly pass prompt data to other parts of the model, potentially causing them to violate guidelines, generate harmful content, enable unauthorized access, or influence critical decisions. While techniques like Retrieval Augmented Generation (RAG) and fine-tuning aim to make LLM outputs more relevant and accurate, research shows that they do not fully mitigate prompt injection vulnerabilities.

While prompt injection and jailbreaking are related concepts in LLM security, they are often used interchangeably. Prompt injection involves manipulating model responses through specific inputs to alter its behavior, which can include bypassing safety measures. Jailbreaking is a form of prompt injection where the attacker provides inputs that cause the model to disregard its safety protocols entirely. Developers can build safeguards into system prompts and input handling to help mitigate prompt injection attacks, but effective prevention of jailbreaking requires ongoing updates to the model's training and safety mechanisms.

Types of Prompt Injection Vulnerabilities

Direct Prompt Injections

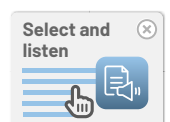
Direct prompt injections occur when a user's prompt input directly alters the behavior of the model in unintended or unexpected ways. The input can be either intentional (i.e., a malicious actor deliberately crafting a prompt to exploit the model) or unintentional (i.e., a user inadvertently providing input that triggers unexpected behavior).

Indirect Prompt Injections

Indirect prompt injections occur when an LLM accepts input from external sources, such as websites or files. The content may have in the external content data that when interpreted by the model, alters the behavior of the model in unintended or unexpected ways. Like direct injections, indirect injections can be either intentional or unintentional.

The severity and nature of the impact of a successful prompt injection attack can vary greatly and are largely dependent on both the business context the model operates in, and the agency with which the model is architected. Generally, however, prompt injection can lead to unintended outcomes, including but not limited to:

- Disclosure of sensitive information
- Revealing sensitive information about AI system infrastructure or system prompts
- Content manipulation leading to incorrect or biased outputs
- Providing unauthorized access to functions available to the LLM
- Executing arbitrary commands in connected systems
- Manipulating critical decision-making processes



The rise of multimodal AI, which processes multiple data types simultaneously, introduces unique prompt injection risks. Malicious actors could exploit interactions between modalities, such as hiding instructions in images that accompany benign text. The complexity of these systems expands the attack surface. Multimodal models may also be susceptible to novel cross-modal attacks that are difficult to detect and mitigate with current techniques. Robust multimodal-specific defenses are an important area for further research and development.

Prevention and Mitigation Strategies

Prompt injection vulnerabilities are possible due to the nature of generative AI. Given the stochastic influence at the heart of the way models work, it is unclear if there are fool-proof methods of prevention for prompt injection. However, the following measures can mitigate the impact of prompt injections:

1. Constrain model behavior

Provide specific instructions about the model's role, capabilities, and limitations within the system prompt. Enforce strict context adherence, limit responses to specific tasks or topics, and instruct the model to ignore attempts to modify core instructions.

2. Define and validate expected output formats

Specify clear output formats, request detailed reasoning and source citations, and use deterministic code to validate adherence to these formats.

3. Implement input and output filtering

Define sensitive categories and construct rules for identifying and handling such content. Apply semantic filters and use string-checking to scan for non-allowed content. Evaluate responses using the RAG Triad: Assess context relevance, groundedness, and question/answer relevance to identify potentially malicious outputs.

4. Enforce privilege control and least privilege access

Provide the application with its own API tokens for extensible functionality, and handle these functions in code rather than providing them to the model. Restrict the model's access privileges to the minimum necessary for its intended operations.

5. Require human approval for high-risk actions

Implement human-in-the-loop controls for privileged operations to prevent unauthorized actions.

6. Segregate and identify external content

Separate and clearly denote untrusted content to limit its influence on user prompts.

7. Conduct adversarial testing and attack simulations \

Perform regular penetration testing and breach simulations, treating the model as an untrusted user to test the effectiveness of trust boundaries and access controls.

Example Attack Scenarios

Scenario #1: Direct Injection

An attacker injects a prompt into a customer support chatbot, instructing it to ignore previous guidelines, query private data stores, and send emails, leading to unauthorized access and privilege escalation.

Scenario #2: Indirect Injection

A user employs an LLM to summarize a webpage containing hidden instructions that cause the LLM to insert an image linking to a URL, leading to exfiltration of the the private conversation.

Scenario #3: Unintentional Injection

A company includes an instruction in a job description to identify AI-generated applications. An applicant, unaware of this instruction, uses an LLM to optimize their resume, inadvertently triggering the AI detection.

Scenario #4: Intentional Model Influence

An attacker modifies a document in a repository used by a Retrieval-Augmented Generation (RAG) application. When a user's query returns the modified content, the malicious instructions alter the LLM's output, generating misleading results.

Scenario #5: Code Injection

An attacker exploits a vulnerability (CVE-2024-5184) in an LLM-powered email assistant to inject malicious prompts, allowing access to sensitive information and manipulation of email content.

Scenario #6: Payload Splitting

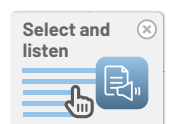
An attacker uploads a resume with split malicious prompts. When an LLM is used to evaluate the candidate, the combined prompts manipulate the model's response, resulting in a positive recommendation despite the actual resume contents.

Scenario #7: Multimodal Injection

An attacker embeds a malicious prompt within an image that accompanies benign text. When a multimodal AI processes the image and text concurrently, the hidden prompt alters the model's behavior, potentially leading to unauthorized actions or disclosure of sensitive information.

Scenario #8: Adversarial Suffix

An attacker appends a seemingly meaningless string of characters to a prompt, which influences the LLM's output



in a malicious way, bypassing safety measures.

Scenario #9: Multilingual/Obfuscated Attack

An attacker uses multiple languages or encodes malicious instructions (e.g., using Base64 or emojis) to evade filters and manipulate the LLM's behavior.

Reference Links

1. ChatGPT Plugin Vulnerabilities – Chat with Code **Embrace the Red**
2. ChatGPT Cross Plugin Request Forgery and Prompt Injection **Embrace the Red**
3. Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection **Arxiv**
4. Defending ChatGPT against Jailbreak Attack via Self-Reminder **Research Square**
5. Prompt Injection attack against LLM-integrated Applications **Cornell University**
6. Inject My PDF: Prompt Injection for your Resume **Kai Greshake**
7. Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection **Cornell University**
8. Threat Modeling LLM Applications **AI Village**
9. Reducing The Impact of Prompt Injection Attacks Through Design **Kudelski Security**
10. Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations (nist.gov)
11. 2407.07403 A Survey of Attacks on Large Vision-Language Models: Resources, Advances, and Future Trends (arxiv.org)
12. Exploiting Programmatic Behavior of LLMs: Dual-Use Through Standard Security Attacks
13. Universal and Transferable Adversarial Attacks on Aligned Language Models (arxiv.org)
14. From ChatGPT to ThreatGPT: Impact of Generative AI in Cybersecurity and Privacy (arxiv.org)

Related Frameworks and Taxonomies

Refer to this section for comprehensive information, scenarios strategies relating to infrastructure deployment, applied environment controls and other best practices.

- AML.T0051.000 – LLM Prompt Injection: Direct **MITRE ATLAS**
- AML.T0051.001 – LLM Prompt Injection: Indirect **MITRE ATLAS**
- AML.T0054 – LLM Jailbreak Injection: Direct **MITRE ATLAS**

LLM Top 10



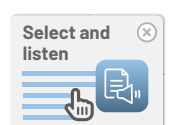
LLM01:2025 Prompt Injection

A Prompt Injection Vulnerability occurs when user prompts alter the LLM's behavior or output in unintended ways. These inputs...



LLM02:2025 Sensitive Information Disclosure

Sensitive information can affect both the LLM and its application context. This includes personal identifiable information (PII), financial details,...





LLM03:2025 Supply Chain

LLM supply chains are susceptible to various vulnerabilities, which can affect the integrity of training data, models, and deployment...

LLM04: 2025

Data and Model Poisoning

LLM04:2025 Data and Model Poisoning

Data poisoning occurs when pre-training, fine-tuning, or embedding data is manipulated to introduce vulnerabilities, backdoors, or biases. This manipulation...

LLM05: 2025

Improper Output Handling

LLM05:2025 Improper Output Handling

Improper Output Handling refers specifically to insufficient validation, sanitization, and handling of the outputs generated by large language models...

LLM06: 2025

Excessive Agency

LLM06:2025 Excessive Agency

An LLM-based system is often granted a degree of agency by its developer – the ability to call functions...

LLM07: 2025

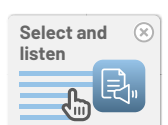
System Prompt Leakage

LLM07:2025 System Prompt Leakage

The system prompt leakage vulnerability in LLMs refers to the risk that the system prompts or instructions used to...

LLM08: 2025

Vector and Embedding





Embedding Weaknesses

LLM08:2025 Vector and Embedding Weaknesses

Vectors and embeddings vulnerabilities present significant security risks in systems utilizing Retrieval Augmented Generation (RAG) with Large Language Models...

LLM09: 2025 Misinformation

LLM09:2025 Misinformation

Misinformation from LLMs poses a core vulnerability for applications relying on these models. Misinformation occurs when LLMs produce false...

LLM10: 2025 Unbounded Consumption

LLM10:2025 Unbounded Consumption

Unbounded Consumption refers to the process where a Large Language Model (LLM) generates outputs based on input queries or...

